

Controllability and Dynamic Programming

Bellman, Riccati, and the Value Function

RCI Lab

Kyung Hee University

February 21, 2026

- LQR via Shooting - LQR as a QP - Riccati Recursion - Infinite-Horizon LQR

Today:

- Controllability - Bellman's Principle - Dynamic Programming - LQR via DP

Controllability

Question:

When does LQR work?

Consider time-invariant system:

$$x_{k+1} = Ax_k + Bu_k$$

Unrolling:

$$x_N = A^N x_0 + \sum_{i=0}^{N-1} A^{N-1-i} Bu_i$$

Stacking inputs:

$$x_N = A^N x_0 + C \begin{bmatrix} u_0 \\ u_1 \\ \vdots \\ u_{N-1} \end{bmatrix}$$

Controllability Matrix

$$\mathcal{C} = [B \quad AB \quad A^2B \quad \dots \quad A^{N-1}B]$$

System is controllable if:

$$\text{rank}(\mathcal{C}) = n$$

where $n = \dim(x)$.

Then any state reachable in finite steps.

Cayley–Hamilton theorem:

$$A^n = \sum_{i=0}^{n-1} \alpha_i A^i$$

Therefore:

Adding powers beyond n cannot increase rank.

Thus:

Check controllability within n steps.

Bellman's Principle of Optimality

Key idea:

Sub-trajectories of optimal trajectories must themselves be optimal.

Past controls affect only future states.

Future controls cannot affect past states.

Therefore:

Optimal control has recursive structure.

Value Function

Define cost-to-go:

$$V_k(x) = \min_{\{u_i\}_{i=k}^{N-1}} \sum_{i=k}^{N-1} \ell(x_i, u_i) + \ell_f(x_N)$$

Bellman equation:

$$V_k(x) = \min_u [\ell(x, u) + V_{k+1}(f(x, u))]$$

Dynamic Programming Algorithm

Initialize:

$$V_N(x) = \ell_f(x)$$

Backward recursion:

$$V_k(x) = \min_u [\ell(x, u) + V_{k+1}(f(x, u))]$$

Optimal policy:

$$u_k(x) = \arg \min [\ell(x, u) + V_{k+1}(f(x, u))]$$

LQR via Dynamic Programming

Assume quadratic value:

$$V_k(x) = x^T P_k x$$

Compute:

$$V_k(x) = \min_u \left[x^T Q x + u^T R u + (Ax + Bu)^T P_{k+1} (Ax + Bu) \right]$$

Differentiate w.r.t u :

$$2Ru + 2B^T P_{k+1}(Ax + Bu) = 0$$

$$(R + B^T P_{k+1} B)u = -B^T P_{k+1} Ax$$

$$u_k = -K_k x$$

$$K_k = (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$

Substitute back:

$$P_k = Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A$$

Same Riccati equation as before.

Thus:

DP = structured solution of LQR QP.

Curse of Dimensionality

DP guarantees global optimality.

But:

Value function defined over state space.

If state dimension = n ,

grid size grows exponentially.

Intractable for nonlinear high-dimensional systems.

Why Do We Care?

- LQR works because quadratic structure preserved. - Nonlinear systems
→ value function not quadratic. - Approximate DP → basis of Reinforcement Learning.
- Modern RL approximates:

$$V(x) \quad \text{or} \quad Q(x, u)$$

Lagrange Multipliers Revisited

From QP derivation:

$$\lambda_k = P_k x_k$$

From DP:

$$V_k(x) = x^T P_k x$$

$$\nabla_x V_k(x) = 2P_k x$$

Thus:

Costate = Gradient of value function.

This generalizes beyond LQR.

Big Picture

Controllability ensures system reachable.

Bellman gives recursive structure.

DP yields Riccati for LQR.

Lagrange multipliers = Value gradients.

This connects:

$\text{KKT} \leftrightarrow \text{Pontryagin} \leftrightarrow \text{DP} \leftrightarrow \text{Riccati}$